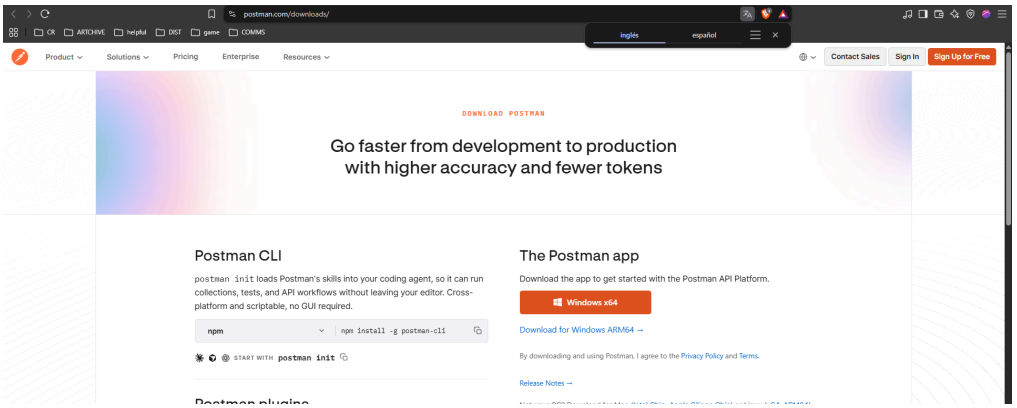
Meneses Calderas Grecia Irais Oscar Manuel Ruiz Hurtado

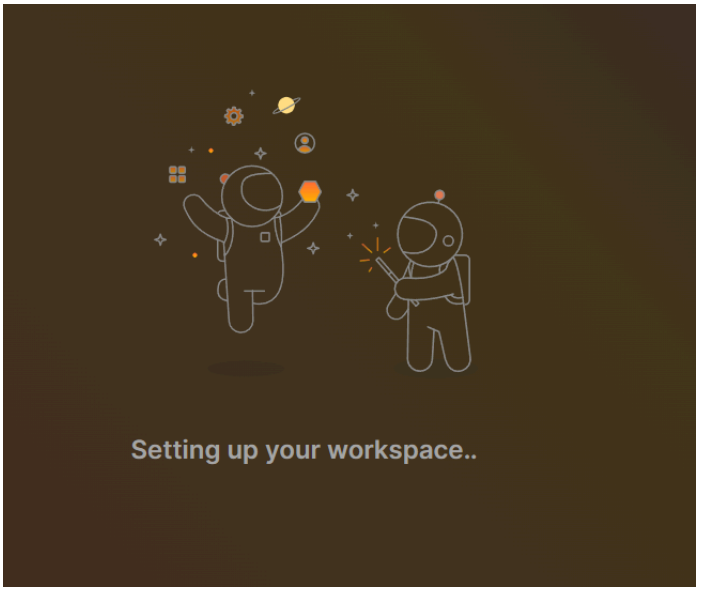
Mexi Banco con Docker Compose: lo de la clase, en tu máquina

Parte 0.

Para poder empezar, se descarga Postman desde su página oficial, conecté en este caso mi cuenta de github porque es la que se estará usando en el curso y configuré la cuenta.







Figuras 0.1., 0.2 y 0.3. Downloading and setting up Postman.

Parte 1.

Se clona el repositorio y nos colocamos dentro del mismo.

Parte 2.

Después, con Docker Desktop abierto en la computadora, primero se ejecuta el comando que nos hará posible leer correctamente los archivos con extensión .mvnw.

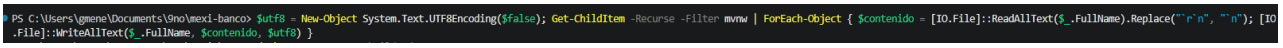


Figura 2.1. Comando para lectura de .mvnw.

Ahora si se construye el compose y se observa que los servicios tengan el estado de healthy.



Figura 2.2. Levantamiento exitoso del servicio

Con el comando `docker compose ps` vemos que se levantaron los servicios y aparecen como healthy.

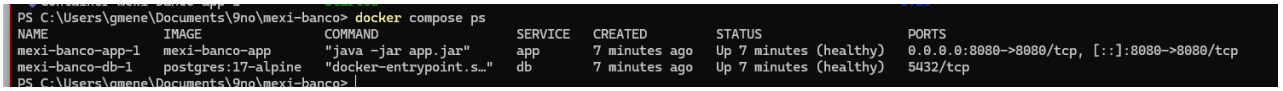


Figura 2.3. Levantamiento exitoso del servicio.

Parte 3.

Se descarga el archivo json al principio del laboratorio y se importa en Postman.

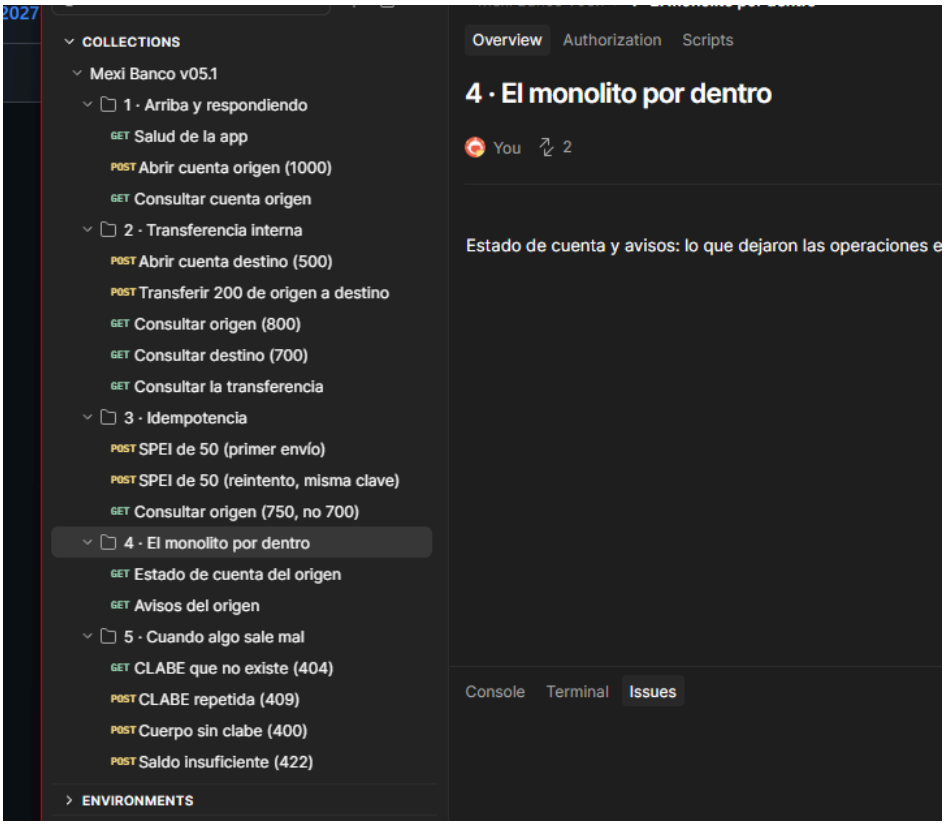


Figura 3.1. Importación exitosa a Postman.

Se corre cada petición en orden.

1. *Arriba y respondiendo*

Empezando con revisar la salud de la app con una petición GET. Se obtiene el código de respuesta 200 OK, y se puede ver que el status está arriba y listo, por lo que se concluye que la app está sana y se puede utilizar.

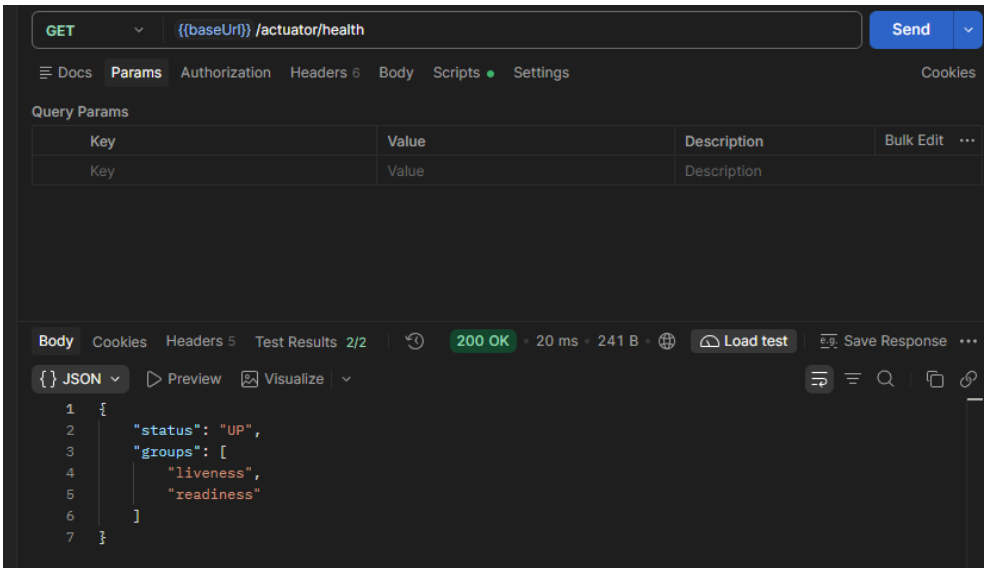


Figura 3.1.1. Servicio está saludable.

La segunda petición es de tipo POST y sirve para crear una cuenta, la de Ana con un saldo inicial de 100 pesos a la que se le asigna un id y una clabe. Nos retornó el código de resultado 201 y con que fue creado exitosamente.

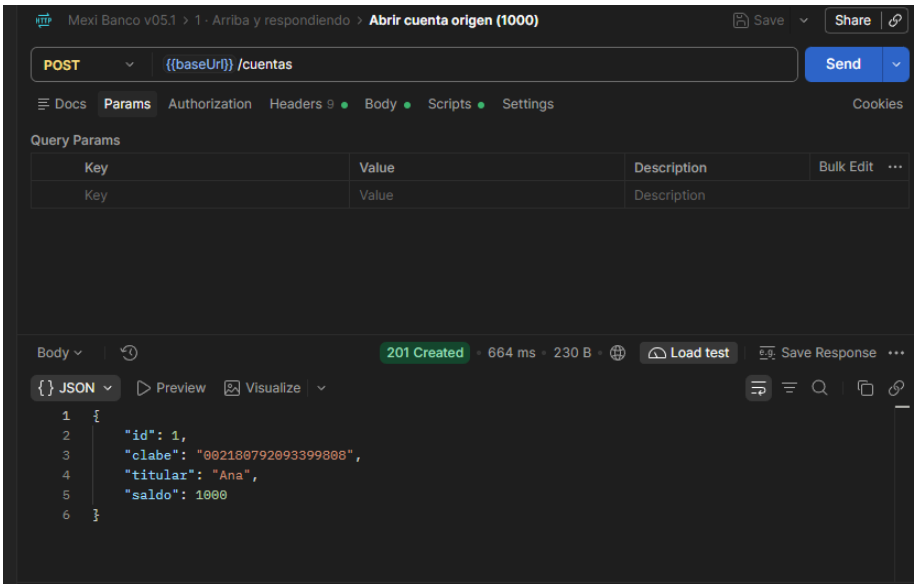


Figura 3.1.2. Creación cuenta 1.

La tercera hace una consulta a las cuentas y se obtiene de resultado el código 200 OK, y los datos de la consulta, con lo que se comprueba que la cuenta se creó correctamente y que ningún dato fue corrompido.

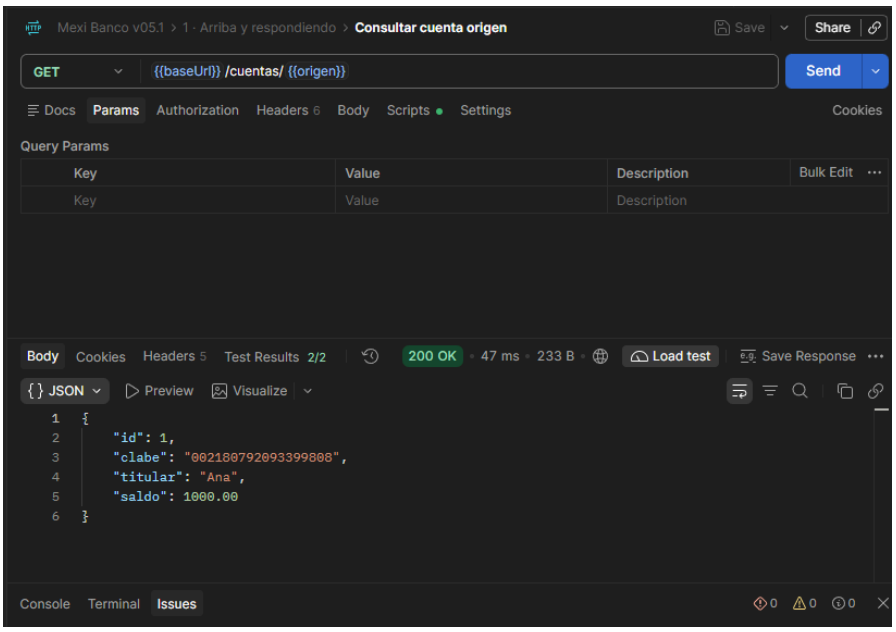


Figura 3.1.3. Consulta exitosa a la cuenta.

2. *Transferencia interna*

Ahora se crea la cuenta de Beto, se crearon las cuenta de ana y beto varias veces porque estaba haciendo troubleshooting de un problema de conexión, y tuve que volver a levantar todo y borrar el workspace pero aun así se crearon.

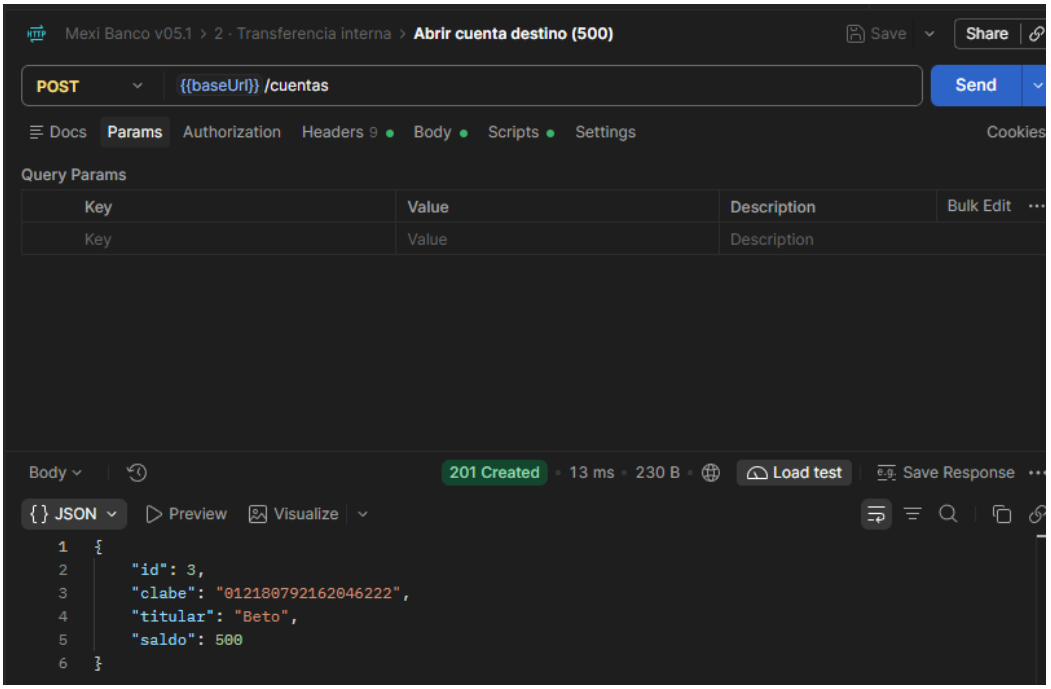


Figura 3.2.1. Creación exitosa cuenta Beto.

Después, se transfiere un monto desde la cuenta origen ana a la cuenta destino beto y se obtiene que se creó la transferencia correctamente con el código 201.

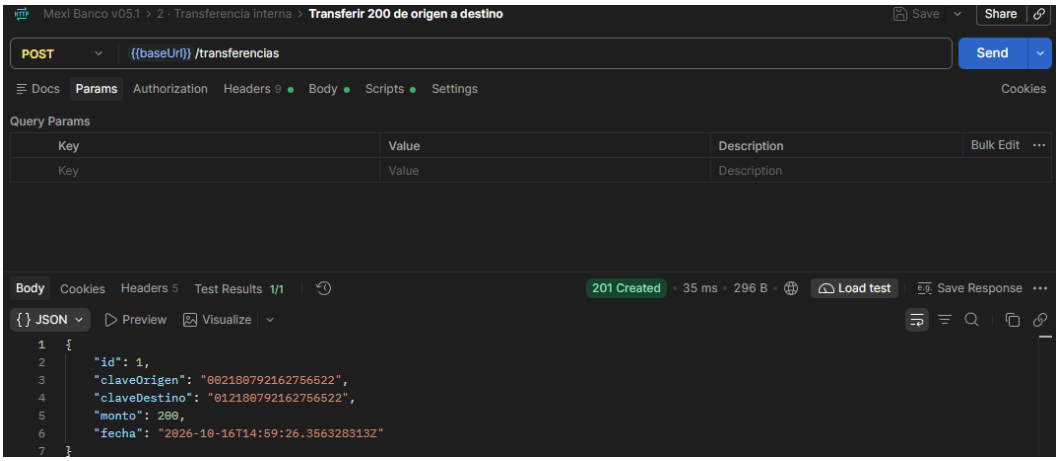


Figura 3.2.2. Transferencia exitosa.

Ahora, se consulta a la cuenta de origen, y se puede ver que se le sustrajeron 200 pesos a los 1000 originales de la transferencia, por lo que se realizó correctamente. Código de ejecución 200 OK.

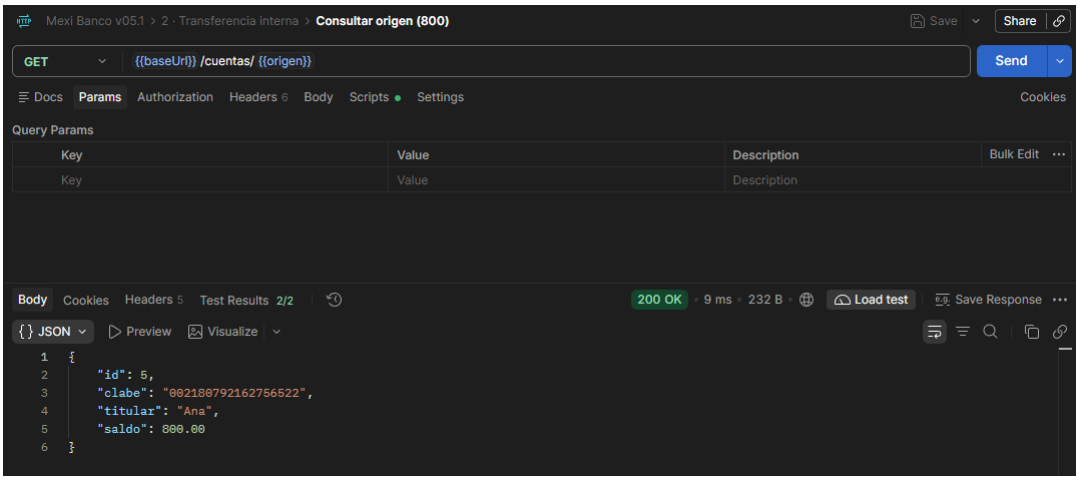


Figura 3.2.3. Consulta confirmando Transferencia exitosa (origen).

También se consulta la cuenta destino que originalmente tenía 500 ahora muestra 700.Código de ejecución 200 OK.

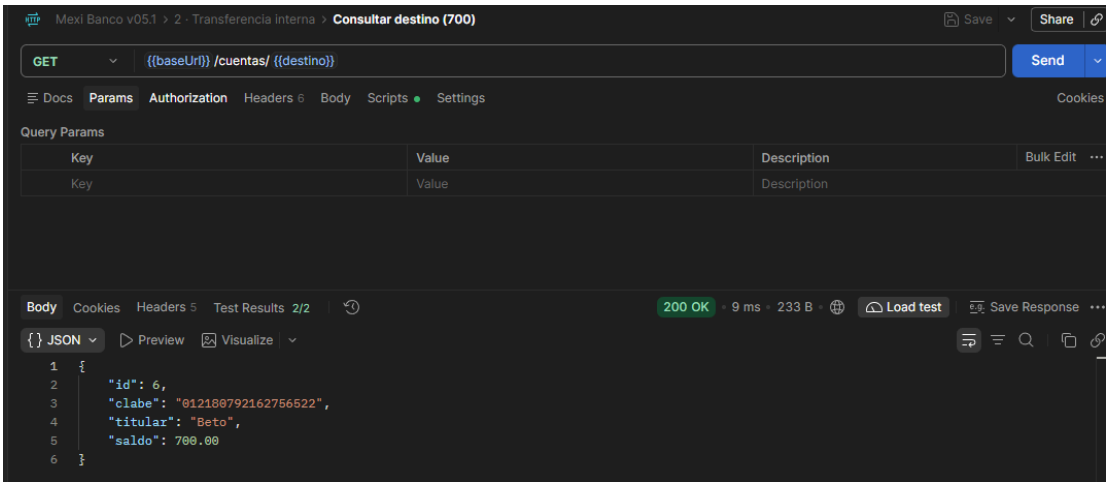


Figura 3.2.4. Consulta confirmando Transferencia exitosa (destino).

Por último, se consulta a la transferencia en la tabla de transferencias y todo parece correcto y existente. Código de resultado de consulta 200 OK.

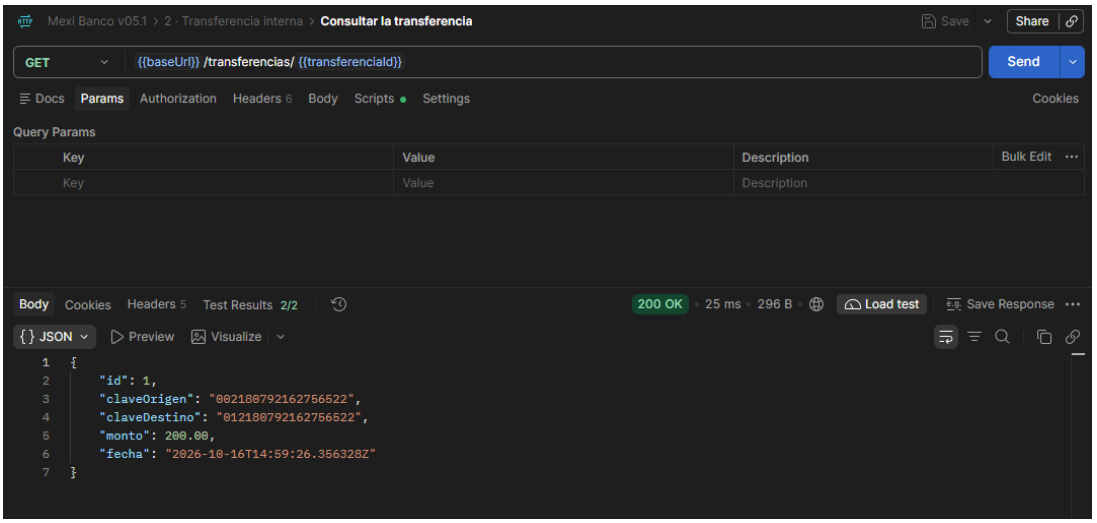


Figura 3.2.5. Consulta confirmando Transferencia exitosa (transferencia).

3. *Idempotencia*

La primera petición de tipo POST envía un SPEI de 50 pesos utilizando idempotency-key. Se obtuvo el código 201 Created de respuesta, por lo que se registró con el estado ENVIADO junto con la especificación del banco.

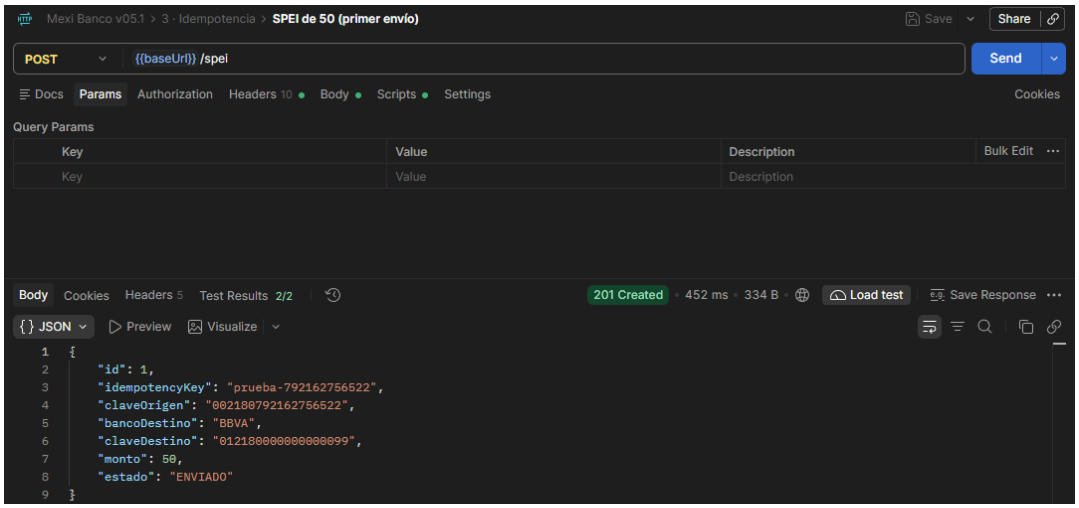


Figura 3.3.1. SPEI enviado.

Después, se reintenta el mismo SPEI, y se realiza con el mismo idempotency-key e incluso tiene el mismo identificador, por lo que sabemos que se repitió la operación pero no se uso otro SPEI. Se obtuvo el mismo código de respuesta.

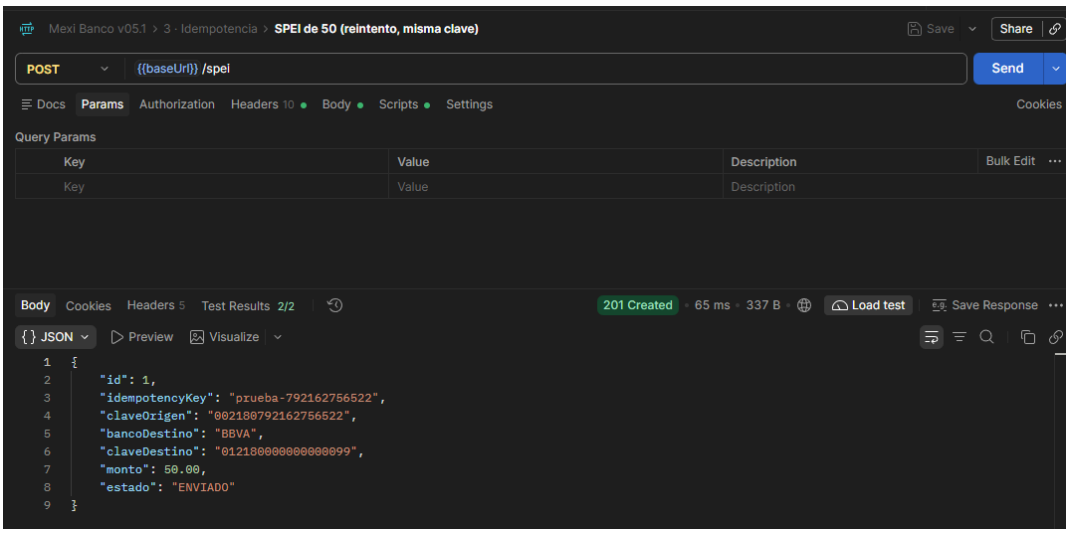


Figura 3.3.2. SPEI repetido.

Después, se vuelve a consultar la cuenta de Ana, la cual realizó el SPEI y con la que se han estado probando los ejercicios, podemos ver que de los 800 que tenía, bajó a 750 pesos después de hacer el SPEI de 50 pesos, por lo que podemos ver que se aplicó el cambio y sólo se aplicó una vez. La consulta devolvió el código 200 OK.

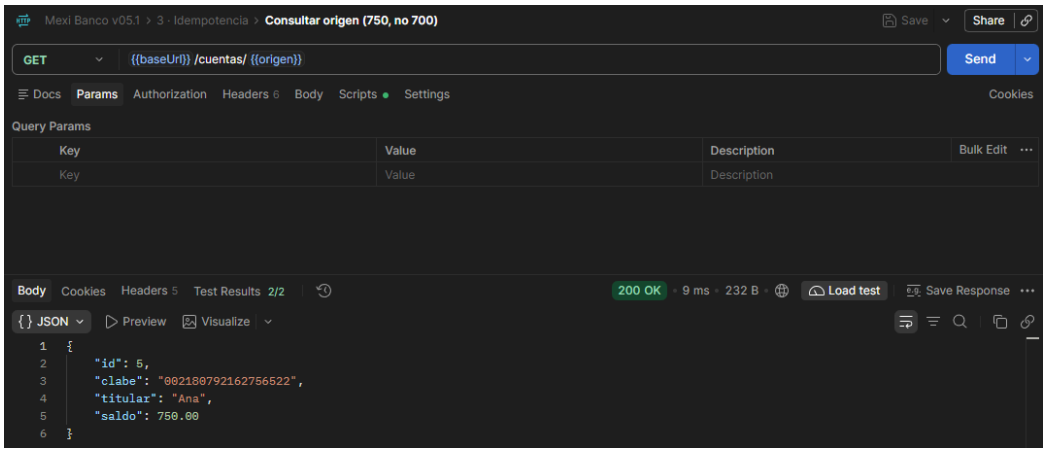


Figura 3.3.3. SPEI aplicado a la cuenta.

4. *El monolito por dentro*

Se hace una consulta a los movimientos de Ana, la cuenta de origen y se pueden ver desglosados por el primer depósito de 1000 pesos, después la transferencia que le quitó 200 y por último el SPEI de 50 pesos. Además de que se puede ver el resultado en su cuenta tras cada operación. Se hizo la consulta con éxito.

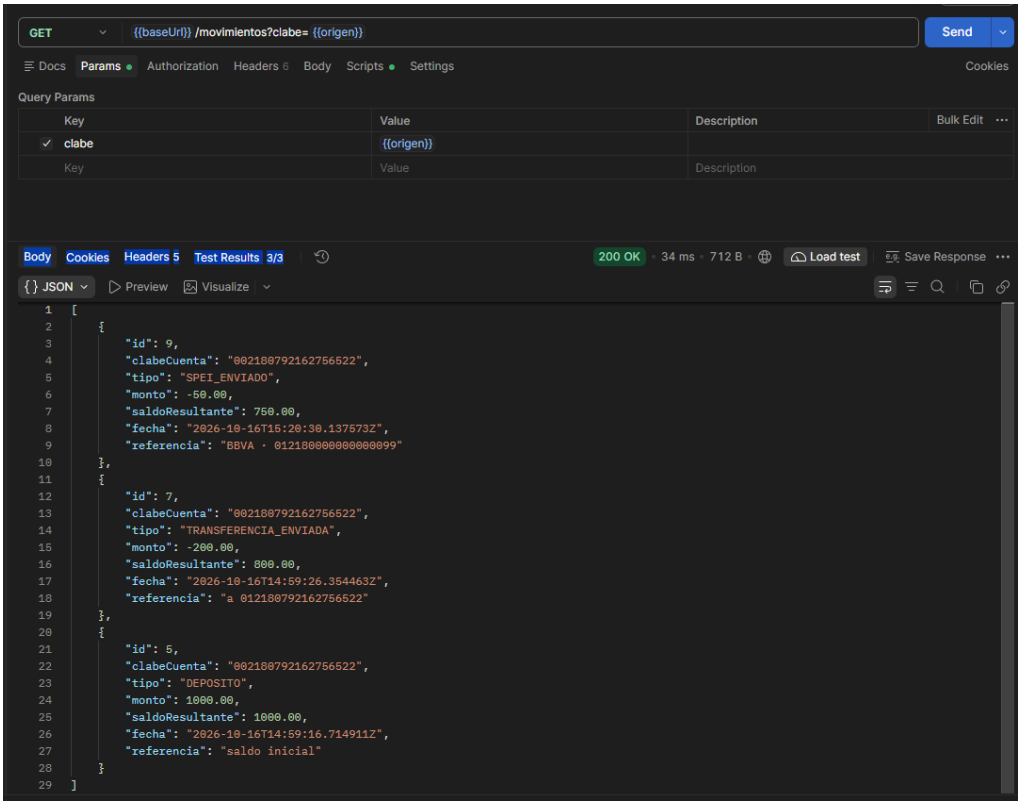


Figura 3.4.1. Movimientos de la cuenta de origen.

Por último en esta categoría, se hace una consulta GET a la tabla de notificaciones asociada a la cuenta de origen, estos tienen la información de la clave, el movimiento que se hizo por la cantidad asociada y la cantidad restante y la fecha para trazabilidad. Código de resultado 200 OK.

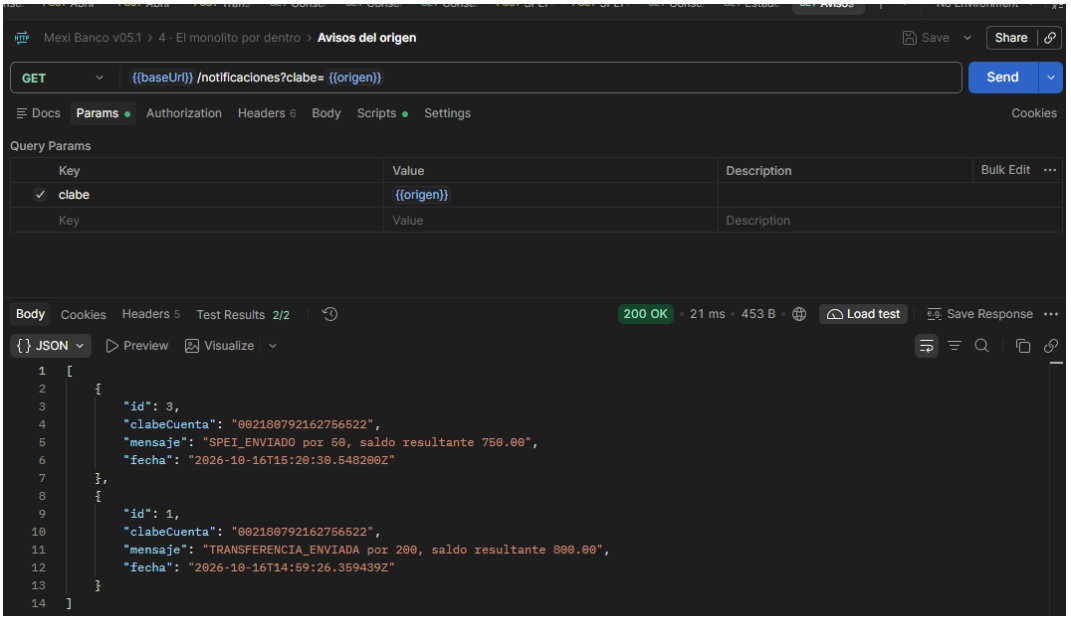


Figura 3.4.2. Notificaciones de la cuenta de origen.

5. Cuando algo sale mal

Se realiza una petición de tipo GET, una consulta para una cuenta con una clabe que al momento no existe. La API nos devuelve el código 404 Not Found.

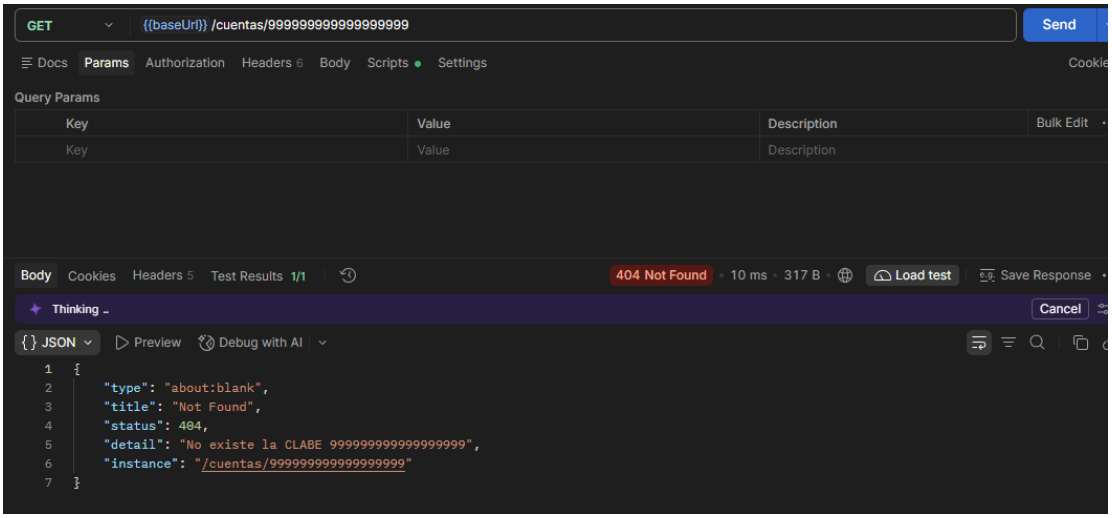


Figura 3.5.1. Cuenta inexistente.

Se intenta crear una cuenta con la misma clabe que otra que ya se creó, la API responde con el código 409 Conflict y en el json se nos informa la razón del conflicto, que es que ya existe una cuenta y no se pueden tener cuentas duplicadas.

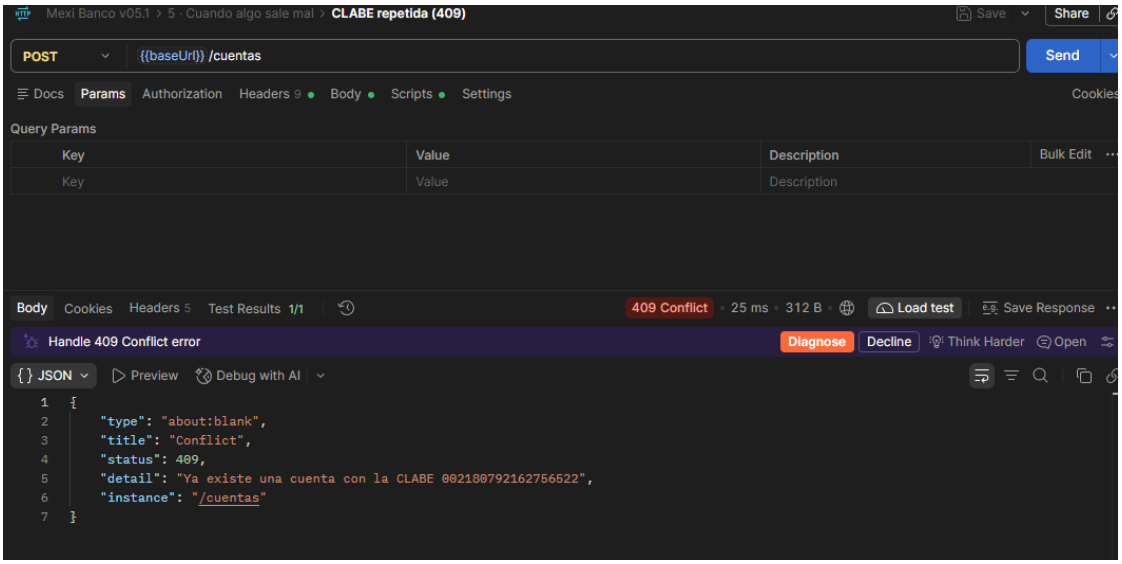


Figura 3.5.2. Duplicación de cuenta.

Ahora se intenta crear una cuenta por medio de una petición POST pero no se envía la clave, por lo que la API responde con el código 400 Bad Request, ya que faltan campos.

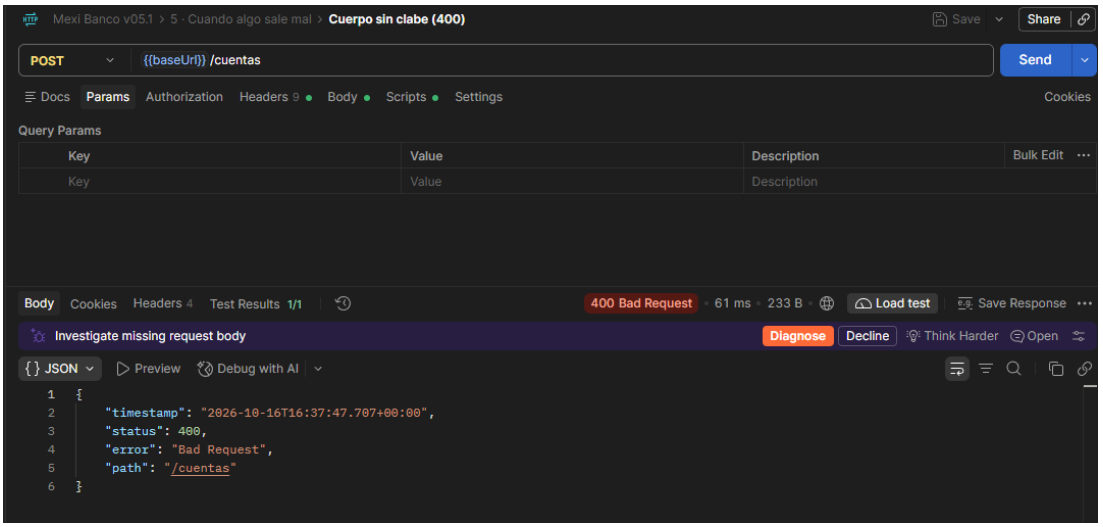


Figura 3.5.3. Petición con campos insuficientes.

Luego, se intenta realizar una transferencia con una cantidad mayor a la disponible en la cuenta de origen, por lo que la API responde con el código 422 Unprocessable Entity y en los detalles se especifica que el saldo es insuficiente para la clabe.

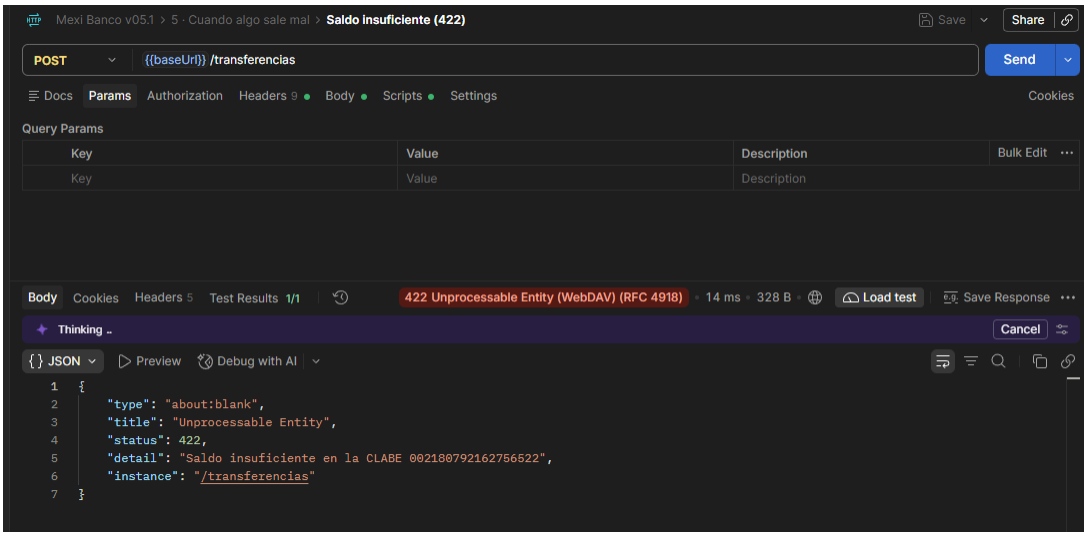


Figura 3.5.4. Transferencia con saldo insuficiente.

Ahora, se intenta realizar una transferencia a sí misma (cuenta). Nos devuelve el código 422 Unprocessable Entity y en los detalles nos explica que es porque se trata de la misma cuenta.

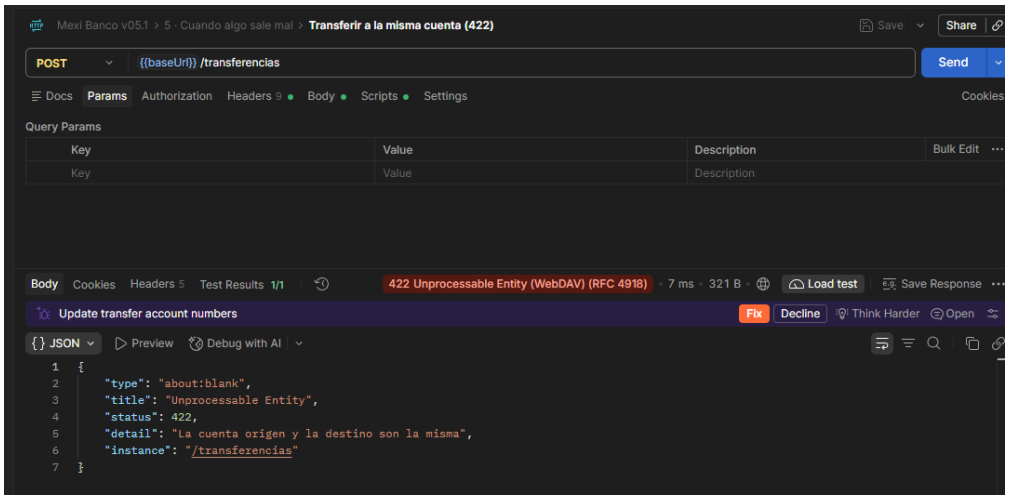


Figura 3.5.5. Transferencia de una cuenta a esa misma.

Se trata de realizar un SPEI pero sin colocar en la petición POST la idempotency-key, por lo que de igual manera cuando no se especificó la clave, la API responde con 400 Bad request.

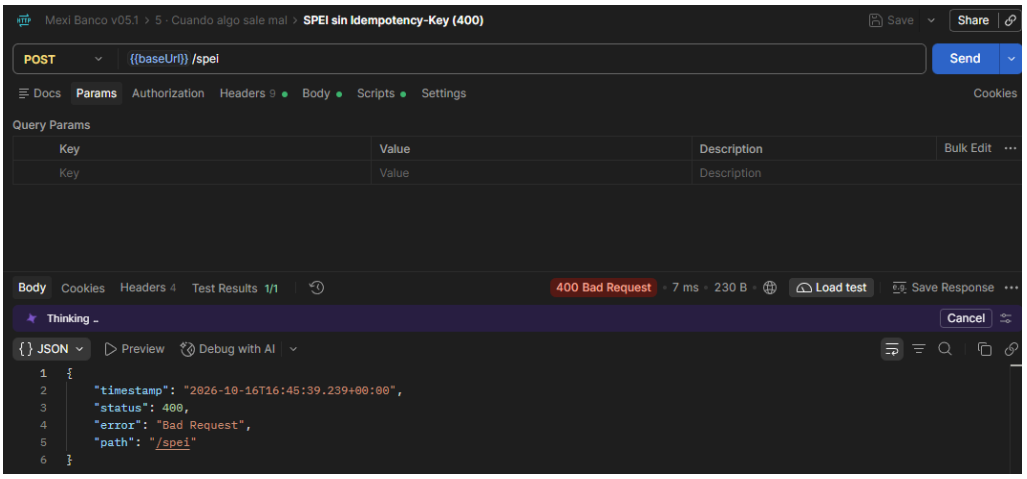


Figura 3.5.6. SPEI sin idempotency-key.

Por último, se verifica que aunque nos haya devuelto errores en cada una de las operaciones, que no haya modificado nada a la cuenta de origen de Ana, el saldo es el mismo y el código es 200 OK.

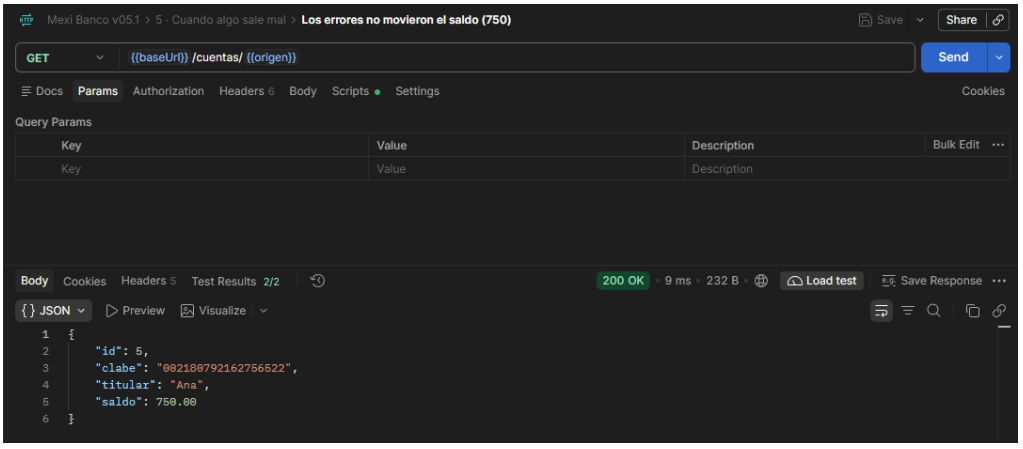


Figura 3.5.7. Consulta de la cuenta de origen sin cambios.

Parte 4.

Como primer paso, se apaga el servicio sin borrar el volumen.

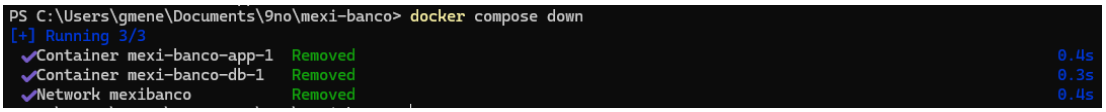


Figura 4.1. Se apaga la aplicación.

Después se hace la última petición en Postman, la que checa si los resultados no cambiaron para la cuenta de Ana, como el servicio está abajo, nos manda un error de conexión.

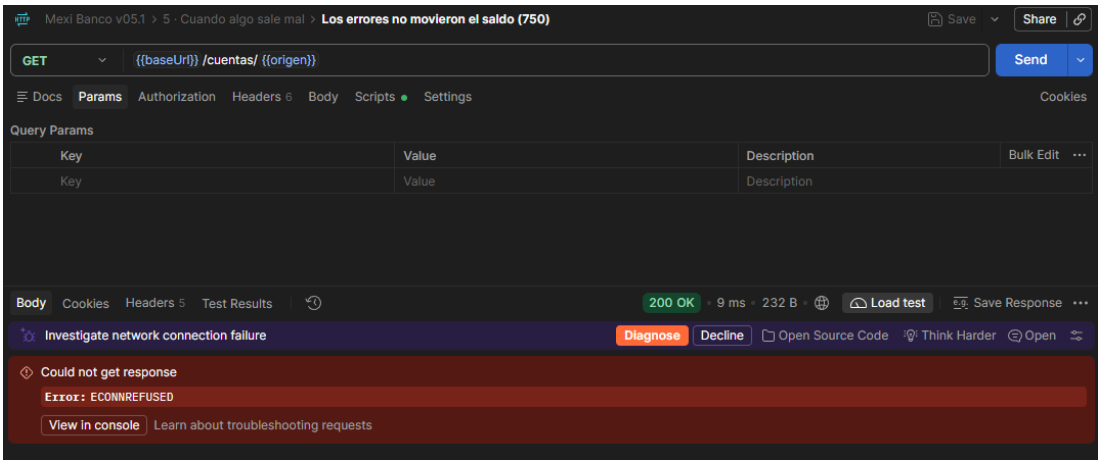


Figura 4.2. Se hace una petición sin conexión (error).

Después se vuelve a encender el servicio.

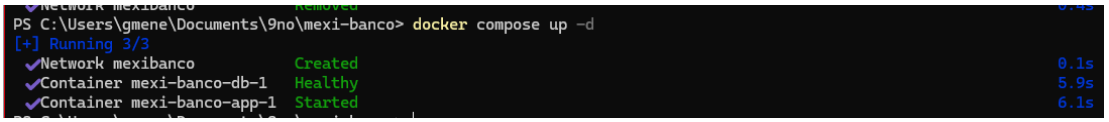


Figura 4.3. Se levanta el servicio.

Como no se borró el volumen, todavía se tiene los datos en la base de datos. Por lo cual al hacer la consulta, se ven los mismos datos, no se ha borrado la cuenta ni se han efectuado cambios.

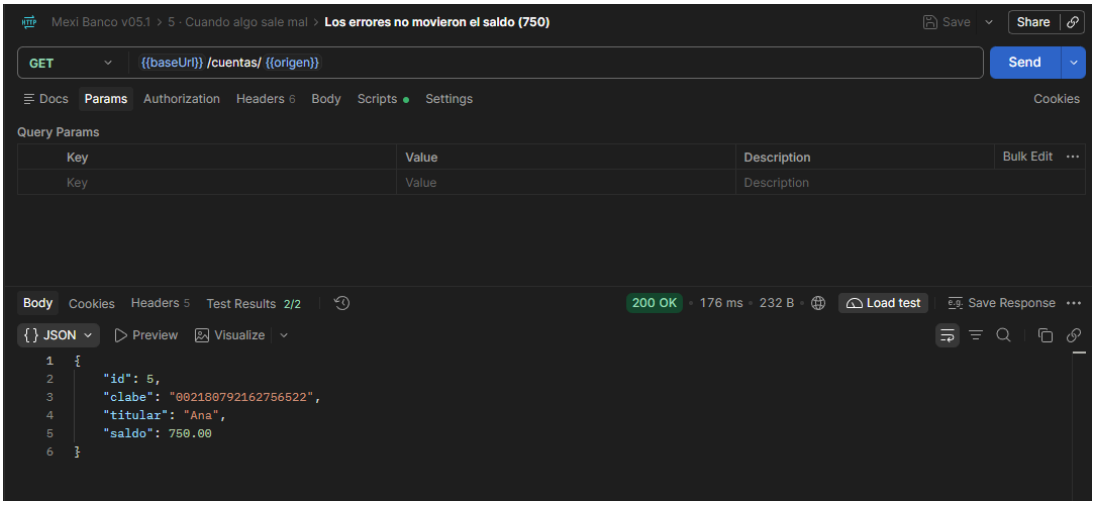


Figura 4.4. Petición de los datos correctos.

Ahora se apaga con todo y volumen.

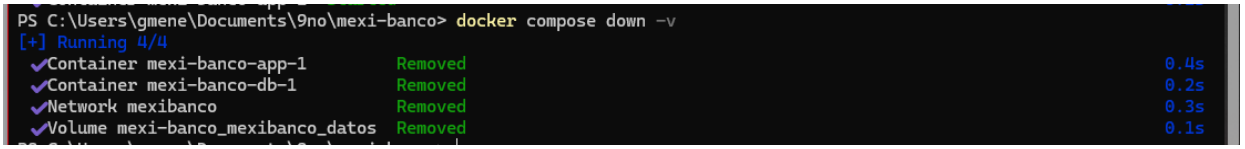


Figura 4.5. Compose down -v.

Se hace la consulta y otra vez nos aparece que no hay conexión.

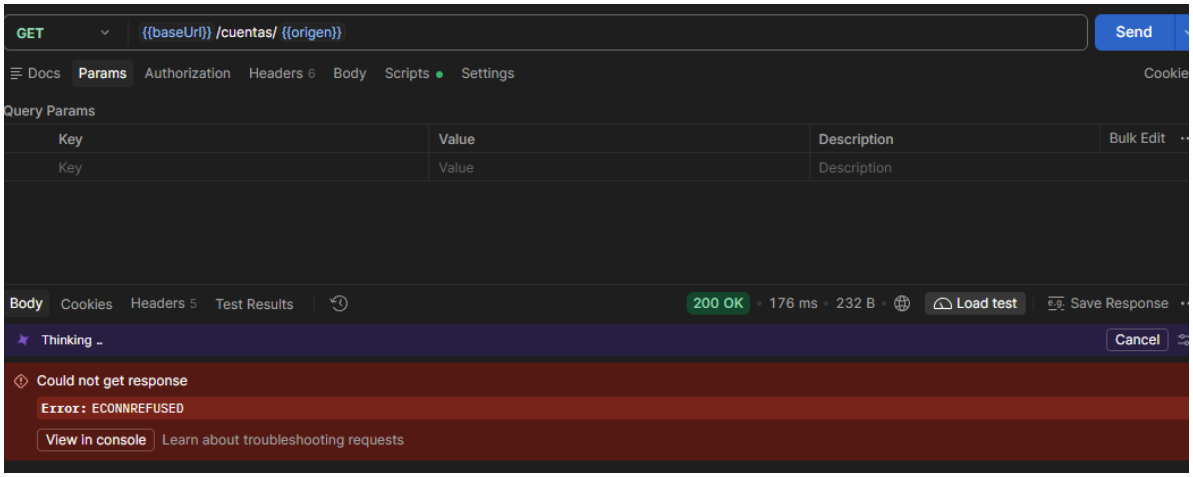


Figura 4.5. Consulta después de apagar el servicio.

Ahora se vuelve a levantar sin el volumen.

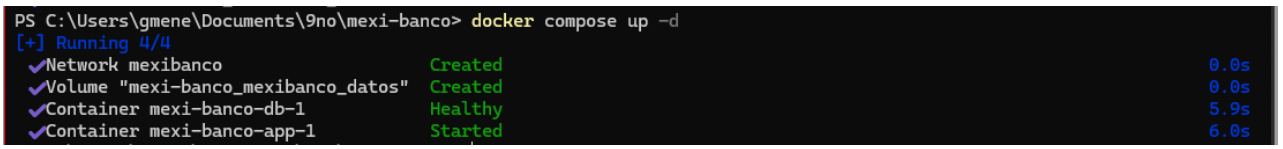


Figura 4.6. Levantamiento del servicio.

Ahora cuando se hace la petición GET de consulta nos manda el error 404 Not Found, ya que los datos se eliminaron con el volumen y con ello las cuentas y todos los movimientos.

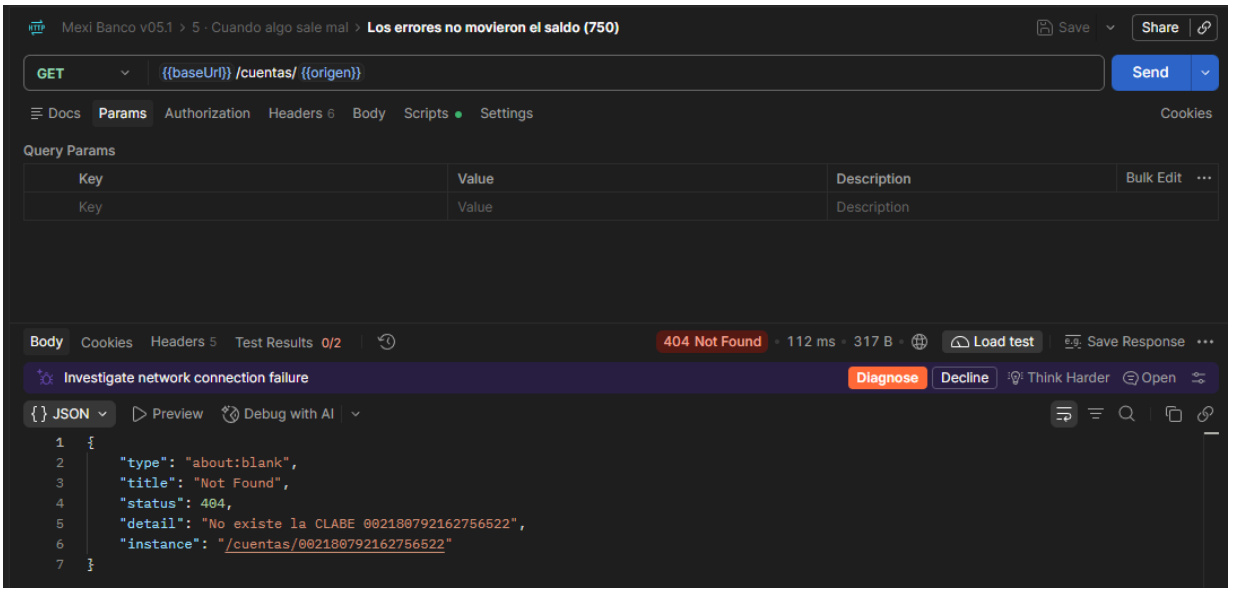


Figura 4.7. Error al hacer la consulta.

Parte 5.

Abre el proyecto en tu editor, en src/main/java/mx/mexibanco. Cada módulo (cuenta, movimiento, transferencia, notificacion y spei) tiene sus cuatro capas: controlador, servicio, repositorio y entidad. Familiarízate con él: es el monolito que vamos a ir partiendo en las siguientes sesiones.

Parte 6.

Léelos línea por línea, como en clase: qué servicios levanta el compose, el healthcheck, la red y el volumen; y en el Dockerfile, sus dos etapas, la que compila y la que ejecuta.

Para el compose levanta los dos servicios que hemos visto en terminal: app y db. Para la base de datos utiliza una imagen de alpine y se le asignan las credenciales así como se le asigna un volumen dentro de postgres. Hace chequeos de salud ejecutando un comando de que esté listo para operar, con intentos y timeout si el servicio no responde dependiendo cada tiempo.

Para app se construye con los archivos que se tienen en las carpetas, y se sube en el puerto 8080. Se crean las credenciales y se especifica que el servicio no puede iniciar o trabajar si no se checa que el servicio de base de datos esté en estado healthy primero. A su vez, también hace un test.

Igualmente, se instancia una red donde estarán los servicios de nombre mexibanco.

```
services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: mexibanco
      POSTGRES_USER: mexibanco
      POSTGRES_PASSWORD: mexibanco
    volumes:
      - mexibanco_datos:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: mexibanco
      DB_USER: mexibanco
      DB_PASSWORD: mexibanco
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP ||
exit 1"]
```

```
      interval: 10s
      timeout: 3s
      retries: 5

networks:
  default:
    name: mexibanco

volumes:
  mexibanco_datos:
```

En el Dockerfile se especifica que la aplicación Java se monta en el entorno de Alpine Linux, el cual es ligero. Se especifica el directorio en el que se trabaja /app dentro del contenedor y a aquí se copian los archivos pom.xml (configuración de Java Maven), mvnw (Maven Wrapper), .mvn (configuración de Maven Wrapper). Después ejecuta Maven dentro de la imagen tomando en cuenta la configuración de dependencias anterior.

Luego, copia la carpeta con el código fuente dentro y por último, compila y empaqueta la aplicación, borrando lo anterior para que no ocurran fallos.

Para la ejecución, define la imagen para el contenedor específica para ejecutar (jre) Alpine Linux. Define el mismo directorio de trabajo. Como en la etapa de compilación se creó un archivo .jar dentro del directorio de app/, ahora lo copia a la imagen final.

Por último, indica que escucha el puerto 8080 y el comando que se ejecutará cuando inicie, que ejecutará la aplicación en java en el entorno de java.

```
# ---- ETAPA 1: CONSTRUCCION ----
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY mvnw .
COPY .mvn .mvn
RUN ./mvnw dependency:go-offline -B
COPY src ./src
RUN ./mvnw clean package -DskipTests

# ---- ETAPA 2: EJECUCION ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Conclusión.

La práctica se pudo realizar sin complicaciones y mostró los resultados esperados de cada una de las ejecuciones de las peticiones, de manera que nos ayudó a conocer el flujo de la aplicación y a cómo está construida por medio de inspeccionar el código.